



Mục tiêu

Sử dụng thư viện spaCy để thực hiện phân tích cú pháp phụ thuộc cho một câu.

Trực quan hóa cây phụ thuộc để hiểu rõ cấu trúc câu.

Truy cập và duyệt (traverse) cây phụ thuộc theo chương trình.

trích xuất thông tin có ý nghĩa từ các mối quan hệ phụ thuộc (ví dụ: tìm chủ ngữ, tân ngữ, bổ ngữ).

Phần 1: Giới thiệu và Cài đặt

```
In [1]: # Cài đặt spaCy nếu chưa có
# !pip install -U spacy
# !python -m spacy download en_core_web_md
```

```
In [2]: import spacy
from spacy import displacy
import pandas as pd
from IPython.display import HTML, display
import warnings
warnings.filterwarnings('ignore')
```

Phần 2: Phân tích câu và Trực quan hóa

2.1. Tải mô hình và phân tích câu

```
In [3]: # Tải mô hình tiếng Anh đã cài đặt
# Sử dụng en_core_web_md vì nó chứa các vector từ và cây cú pháp đầy đủ
## !python -m spacy download en_core_web_md

try:
    nlp = spacy.load("en_core_web_md")
    print("Mô hình 'en_core_web_md' đã được tải thành công!")
except:
    print("Mô hình chưa được tải. Vui lòng chạy lệnh:")

# Câu ví dụ
text = "The quick brown fox jumps over the lazy dog."

# Phân tích câu với pipeline của spaCy
doc = nlp(text)

print(f"Câu gốc: '{text}'")
print(f"\nSố lượng tokens: {len(doc)}")
print(f"\nTokens:")
```

```
for token in doc:
    print(f"    '{token.text}'")
```

Mô hình 'en_core_web_md' đã được tải thành công!

Câu gốc: 'The quick brown fox jumps over the lazy dog.'

Số lượng tokens: 10

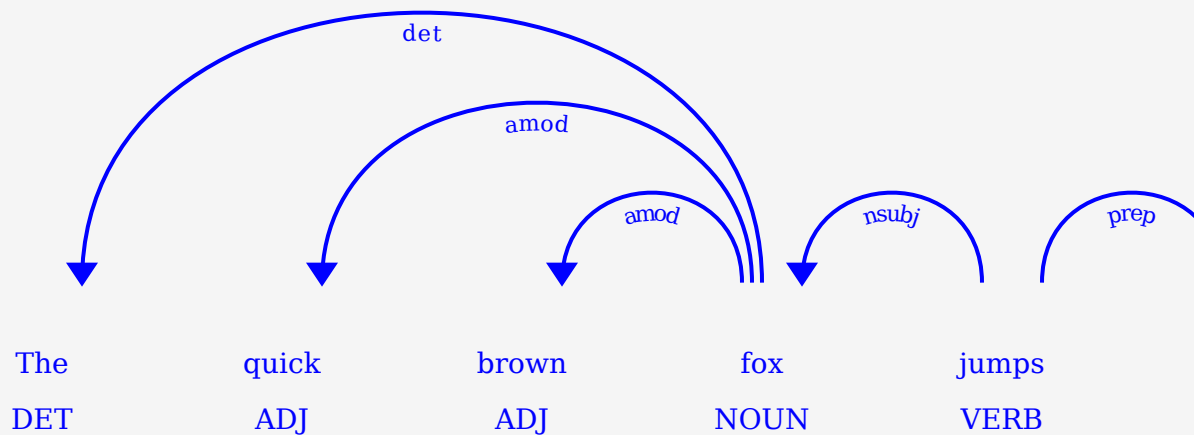
Tokens:

```
'The'
'quick'
'brown'
'fox'
'jumps'
'over'
'the'
'lazy'
'dog'
','
```

2.2. Trực quan hóa cây phụ thuộc trong notebook

```
In [4]: # Hiển thị cây phụ thuộc trực tiếp trong notebook
options = {
    "compact": False,
    "color": "blue",
    "bg": "#f5f5f5",
    "font": "Arial",
    "distance": 120
}

# Hiển thị cây phụ thuộc
displacy.render(doc, style="dep", options=options, jupyter=True)
```



Phần 2.2: Trả lời câu hỏi từ việc trực quan hóa Với câu: "The quick brown fox jumps over the lazy dog."

Từ nào là gốc (ROOT) của câu?

Đáp án: jumps

Giải thích: Trong cây phụ thuộc, ROOT là động từ chính (main verb) của mệnh đề. Ở đây, hành động chính là "jumps".

jumps có những từ phụ thuộc (dependent) nào? Các quan hệ đó là gì?

Đáp án:

fox - Quan hệ nsubj (nominal subject): Chủ ngữ thực hiện hành động "jumps".

over - Quan hệ prep (prepositional modifier): Giới từ bổ nghĩa cho động từ.

. - Quan hệ punct (punctuation): Dấu câu kết thúc câu.

Giải thích: jumps là head, điều khiển ba dependent trên.

fox là head của những từ nào?

Đáp án: The, quick, brown

Quan hệ và giải thích:

The - Quan hệ det (determiner): Mạo từ xác định cho danh từ "fox".

quick - Quan hệ amod (adjectival modifier): Tính từ bổ nghĩa cho danh từ.

brown - Quan hệ amod (adjectival modifier): Tính từ bổ nghĩa cho danh từ.

```
In [5]: # Kiểm tra câu trả lời

# Tìm ROOT
root_token = [token for token in doc if token.dep_ == "ROOT"][0]
print(f"1. ROOT của câu: '{root_token.text}'")

# Kiểm tra dependents của "jumps"
jumps_token = [token for token in doc if token.text == "jumps"][0]
print(f"\n2. Dependents của 'jumps':")
for child in jumps_token.children:
    print(f"    - '{child.text}' với quan hệ: {child.dep_}")

# Kiểm tra children của "fox"
fox_token = [token for token in doc if token.text == "fox"][0]
print(f"\n3. 'fox' là head của:")
for child in fox_token.children:
```

```
print(f"    - '{child.text}' với quan hệ: {child.dep_}")
```

1. ROOT của câu: 'jumps'
2. Dependents của 'jumps':
 - 'fox' với quan hệ: nsubj
 - 'over' với quan hệ: prep
 - '.' với quan hệ: punct
3. 'fox' là head của:
 - 'The' với quan hệ: det
 - 'quick' với quan hệ: amod
 - 'brown' với quan hệ: amod

Phần 3: Truy cập các thành phần trong cây phụ thuộc

```
In [6]: # Phân tích câu mới
text = "Apple is looking at buying U.K. startup for $1 billion"
doc = nlp(text)

print(f"Câu: '{text}'")
print("="*70)
print(f"{'TEXT':<12} | {'DEP':<10} | {'HEAD TEXT':<12} | {'HEAD POS':<8} | {'C'
print("-"*70)

for token in doc:
    # Trích xuất các thuộc tính
    children = [child.text for child in token.children]
    print(f"{'token.text':<12} | {'token.dep_':<10} | {'token.head.text':<12} | {'tok
```

Câu: 'Apple is looking at buying U.K. startup for \$1 billion'

```
=====
TEXT          | DEP          | HEAD TEXT    | HEAD POS    | CHILDREN
-----
Apple         | nsubj        | looking      | VERB        | []
is            | aux          | looking      | VERB        | []
looking       | ROOT         | looking      | VERB        | ['Apple', 'is', 'at']
at            | prep         | looking      | VERB        | ['buying']
buying        | pcomp        | at           | ADP         | ['startup']
U.K.          | compound     | startup      | NOUN        | []
startup       | dobj         | buying       | VERB        | ['U.K.', 'for']
for           | prep         | startup      | NOUN        | ['billion']
$             | quantmod     | billion      | NUM         | []
1             | compound     | billion      | NUM         | []
billion       | pobj         | for          | ADP         | ['$ ', '1']
```

Giải thích các thuộc tính quan trọng:

token.text: Văn bản của token

token.dep_: Nhãn quan hệ phụ thuộc của token này với head của nó

token.head.text: Văn bản của token head

token.head.pos_: Part-of-Speech tag của token head

token.children: Các token con (dependent) của token hiện tại

```
In [7]: # Hiển thị thêm thông tin POS tags
print(f"\n{'TEXT':<12} | {'POS':<8} | {'TAG':<8} | {'LEMMA':<12} | {'DEP':<10}")
print("-"*60)

for token in doc:
    print(f"{'token.text':<12} | {'token.pos_':<8} | {'token.tag_':<8} | {'token.lemma':<12} | {'token.dep_':<10}")
```

TEXT	POS	TAG	LEMMA	DEP
Apple	PROPN	NNP	Apple	nsubj
is	AUX	VBZ	be	aux
looking	VERB	VBG	look	ROOT
at	ADP	IN	at	prep
buying	VERB	VBG	buy	pcomp
U.K.	PROPN	NNP	U.K.	compound
startup	NOUN	NN	startup	dobj
for	ADP	IN	for	prep
\$	SYM	\$	\$	quantmod
1	NUM	CD	1	compound
billion	NUM	CD	billion	pobj

Phần 4: Duyệt cây phụ thuộc để trích xuất thông tin

4.1. Bài toán: Tìm chủ ngữ và tân ngữ của một động từ

```
In [8]: text = "The cat chased the mouse and the dog watched them."
doc = nlp(text)

print(f"Câu: '{text}'")
for token in doc:
    # Chỉ tìm các động từ
    if token.pos_ == "VERB":
        verb = token.text
        subject = ""
        obj = ""

        # Tìm chủ ngữ (nsubj) và tân ngữ (dobj) trong các con của động từ
        for child in token.children:
            if child.dep_ == "nsubj":
                subject = child.text
            if child.dep_ == "dobj":
                obj = child.text

        if subject and obj:
            print(f"Found Triplet: ({subject}, {verb}, {obj})")
        elif subject:
```

```
Câu: 'The cat chased the mouse and the dog watched them.'  
Found Triplet: (cat, chased, mouse)  
Found Triplet: (dog, watched, them)
```

```
graph TD
    S --> NP1[NP]
    S --> VP[VP]
    NP1 --> det1[det]
    NP1 --> nsubj[nsubj]
    VP --> V[V]
    VP --> Sbar[S-bar]
    Sbar --> cc[cc]
    Sbar --> S[S]
    S --> NP2[NP]
    S --> conj[conj]
    NP2 --> dobj[dobj]
    NP2 --> N1[N]
    N1 --> det2[det]
    N1 --> N2[N]
```

The cat chased the mouse and

DET NOUN VERB DET NOUN CCONJ

```
In [10]: text = "The big, fluffy white cat is sleeping on the warm mat."
doc = nlp(text)

print(f"Câu: '{text}'")

for token in doc:
    # Chỉ tìm các danh từ
    if token.pos_ == "NOUN":
        adjectives = []
        articles = []

        # Tìm các tính từ bổ nghĩa (amod) và mạo từ (det) trong các con của danh từ
        for child in token.children:
            if child.dep_ == "amod":
                adjectives.append(child.text)
            elif child.dep_ == "det":
                articles.append(child.text)

        if adjectives:
```

```
print(f"Danh từ '{token.text}' được bổ nghĩa bởi:")
if articles:
    print(f"    Mạo từ: {articles}")
print(f"    Tính từ: {adjectives}")
print()
```

Câu: 'The big, fluffy white cat is sleeping on the warm mat.'

Danh từ 'cat' được bổ nghĩa bởi:

Mạo từ: ['The']

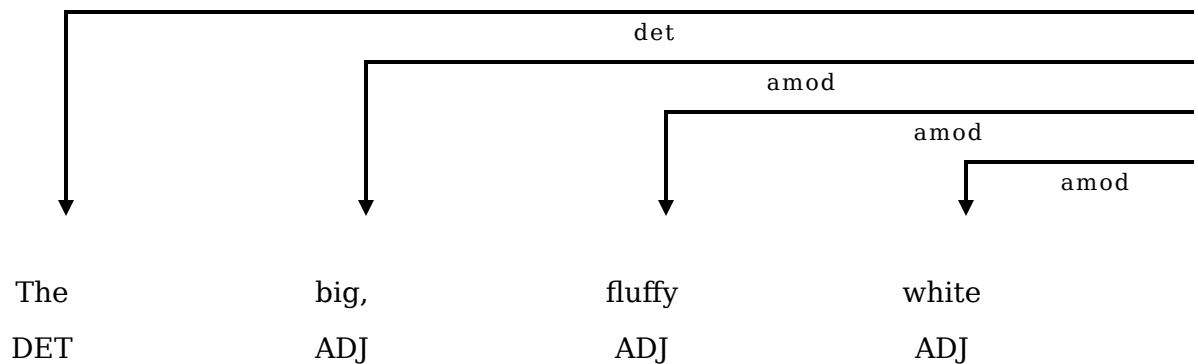
Tính từ: ['big', 'fluffy', 'white']

Danh từ 'mat' được bổ nghĩa bởi:

Mạo từ: ['the']

Tính từ: ['warm']

```
In [11]: # Trực quan hóa
displacy.render(doc, style="dep", options={"compact": True}, jupyter=True)
```



Phần 5: Bài tập tự luyện

Giới thiệu

Phân tích cú pháp phụ thuộc (Dependency Parsing) là một kỹ thuật quan trọng trong xử lý ngôn ngữ tự nhiên, cho phép chúng ta hiểu cấu trúc ngữ pháp của câu thông qua các mối quan hệ phụ thuộc giữa các từ. Trong phần bài tập này, chúng

ta sẽ thực hành ba bài toán cơ bản liên quan đến phân tích cú pháp phụ thuộc sử dụng thư viện spaCy.

5.1. Tìm động từ chính của câu

Viết hàm `find_main_verb(doc)` nhận vào một đối tượng `Doc` của spaCy và trả về `Token` là động từ chính của câu.

Trong phân tích cú pháp phụ thuộc, động từ chính của câu thường có quan hệ phụ thuộc là `ROOT`. Đây là từ trung tâm mà tất cả các từ khác trong câu đều phụ thuộc trực tiếp hoặc gián tiếp vào nó.

```
In [14]: def find_main_verb(doc):
    """
    Tìm và trả về động từ chính của câu (token có quan hệ ROOT)

    Args:
        doc: Đối tượng Doc của spaCy

    Returns:
        Token: Động từ chính của câu, hoặc None nếu không tìm thấy
    """
    # Tìm token có quan hệ phụ thuộc là ROOT
    for token in doc:
        if token.dep_ == "ROOT":
            return token
    return None

# Kiểm tra hàm với các câu khác nhau
test_sentences = [
    "The cat chased the mouse.",
    "She is reading an interesting book.",
    "Quickly, the man ran to the store.",
    "What are you doing?", # Câu hỏi
    "Stop!"
]

print("KIỂM TRA HÀM find_main_verb")

for i, sentence in enumerate(test_sentences, 1):
    doc = nlp(sentence)
    main_verb = find_main_verb(doc)

    print(f"\n{i}. Câu: '{sentence}'")
    if main_verb:
        print(f"Động từ chính: '{main_verb.text}' (POS: {main_verb.pos_}, DEP: {main_verb.dep_})")
        print(f"Các dependents của '{main_verb.text}':")
        for child in main_verb.children:
            print(f"    - '{child.text}' ({child.dep_})")
    else:
        print("Không tìm thấy động từ chính")
```

KIỂM TRA HÀM find_main_verb

1. Câu: 'The cat chased the mouse.'

Động từ chính: 'chased' (POS: VERB, DEP: ROOT)

Các dependents của 'chased':

- 'cat' (nsubj)
- 'mouse' (dobj)
- '.' (punct)

2. Câu: 'She is reading an interesting book.'

Động từ chính: 'reading' (POS: VERB, DEP: ROOT)

Các dependents của 'reading':

- 'She' (nsubj)
- 'is' (aux)
- 'book' (dobj)
- '.' (punct)

3. Câu: 'Quickly, the man ran to the store.'

Động từ chính: 'ran' (POS: VERB, DEP: ROOT)

Các dependents của 'ran':

- 'Quickly' (advmod)
- ',' (punct)
- 'man' (nsubj)
- 'to' (prep)
- '.' (punct)

4. Câu: 'What are you doing?'

Động từ chính: 'doing' (POS: VERB, DEP: ROOT)

Các dependents của 'doing':

- 'What' (dobj)
- 'are' (aux)
- 'you' (nsubj)
- '?' (punct)

5. Câu: 'Stop!'

Động từ chính: 'Stop' (POS: VERB, DEP: ROOT)

Các dependents của 'Stop':

- '!' (punct)

bài 5.2

Tự viết hàm trích xuất cụm danh từ thay vì sử dụng thuộc tính .noun_chunks có sẵn của spaCy.

Cách tiếp cận

Có hai cách tiếp cận chính:

Duyệt từ gốc: Bắt đầu từ danh từ chính và thu thập tất cả các từ trong cây con (subtree) tương ứng hàm `def extract_noun_chunks_custom(doc):`

Duyệt từ modifiers: Tìm tất cả các từ bổ nghĩa trước danh từ tương ứng hàm `def extract_noun_chunks_simple(doc):`

```
In [ ]: def extract_noun_chunks_custom(doc):
        """
        Trích xuất các cụm danh từ từ câu (tự viết thay vì dùng .noun_chunks)

        Args:
            doc: Đối tượng Doc của spaCy

        Returns:
            list: Danh sách các cụm danh từ, mỗi cụm là một danh sách token
        """
        noun_chunks = []
        visited_tokens = set() # Để tránh trùng lặp

        for token in doc:
            # Nếu token là danh từ và chưa được xử lý
            if token.pos_ in ["NOUN", "PROPN"] and token not in visited_tokens:
                # Tìm root của cụm danh từ (có thể là token này hoặc head của nó)
                chunk_root = token

                # Đi ngược lên để tìm root của cụm
                while (chunk_root.head.pos_ in ["NOUN", "PROPN", "ADJ", "DET", "NUM"] and
                       chunk_root.dep_ in ["compound", "amod", "nummod", "det"] and
                       chunk_root.head != chunk_root):
                    chunk_root = chunk_root.head

                # Thu thập toàn bộ subtree của chunk_root
                chunk_tokens = list(chunk_root.subtree)

                # Sắp xếp các token theo thứ tự xuất hiện trong câu
                chunk_tokens_sorted = sorted(chunk_tokens, key=lambda t: t.i)

                # Đánh dấu các token đã được xử lý
                for t in chunk_tokens_sorted:
                    visited_tokens.add(t)

                noun_chunks.append(chunk_tokens_sorted)

        return noun_chunks

def extract_noun_chunks_simple(doc):
    """
    Phiên bản đơn giản hơn: Tìm các danh từ và thu thập modifiers của chúng
    """
    noun_chunks = []

    for token in doc:
        if token.pos_ in ["NOUN", "PROPN"]:
```

```

# Thu thập modifiers đi trước danh từ
modifiers_before = []

# Tìm các modifiers (det, amod, compound) đi trước
# Bằng cách duyệt từ token trở về trước trong câu
current_idx = token.i - 1
while current_idx >= 0:
    current_token = doc[current_idx]

    # Kiểm tra nếu current_token là modifier của token
    is_modifier = False
    for child in token.children:
        if child.i == current_idx and child.dep_ in ["det", "amod":
            is_modifier = True
            break

    # Kiểm tra nếu token là head của current_token
    if not is_modifier:
        for child in current_token.children:
            if child.i == token.i and child.dep_ in ["det", "amod":
                is_modifier = True
                break

    if is_modifier:
        modifiers_before.append(current_token)
        current_idx -= 1
    else:
        break

# Sắp xếp modifiers theo thứ tự xuất hiện
modifiers_before.reverse()

# Tạo cụm danh từ
chunk_tokens = modifiers_before + [token]
noun_chunks.append(chunk_tokens)

return noun_chunks

```

```

In [20]: test_sentences_noun = [
    "The big red car drove quickly.",
    "Three little pigs built their houses.",
    "Apple, Google and Microsoft are tech giants.",
    "The CEO of the company announced new products."
]

print("\nKIỂM TRA HÀM TRÍCH XUẤT CỤM DANH TỪ")

for i, sentence in enumerate(test_sentences_noun, 1):
    doc = nlp(sentence)

    print(f"\n{i}. Câu: '{sentence}'")

    # Phương pháp built-in của spaCy

```

```
print("\n 1. Sử dụng .noun_chunks (built-in):")
for chunk in doc.noun_chunks:
    print(f"      - '{chunk.text}'")

# Phương pháp custom
print("\n 2. Sử dụng hàm extract_noun_chunks_custom:")
custom_chunks = extract_noun_chunks_custom(doc)
for chunk in custom_chunks:
    chunk_text = " ".join([t.text for t in chunk])
    print(f"      - '{chunk_text}'")

# Phương pháp đơn giản
print("\n 3. Sử dụng hàm extract_noun_chunks_simple:")
simple_chunks = extract_noun_chunks_simple(doc)
for chunk in simple_chunks:
    chunk_text = " ".join([t.text for t in chunk])
    print(f"      - '{chunk_text}'")
```

KIỂM TRA HÀM TRÍCH XUẤT CỤM DANH TỪ

1. Câu: 'The big red car drove quickly.'

1. Sử dụng `.noun_chunks` (built-in):
 - 'The big red car'
2. Sử dụng hàm `extract_noun_chunks_custom`:
 - 'The big red car'
3. Sử dụng hàm `extract_noun_chunks_simple`:
 - 'The big red car'

2. Câu: 'Three little pigs built their houses.'

1. Sử dụng `.noun_chunks` (built-in):
 - 'Three little pigs'
 - 'their houses'
2. Sử dụng hàm `extract_noun_chunks_custom`:
 - 'Three little pigs'
 - 'their houses'
3. Sử dụng hàm `extract_noun_chunks_simple`:
 - 'Three little pigs'
 - 'houses'

3. Câu: 'Apple, Google and Microsoft are tech giants.'

1. Sử dụng `.noun_chunks` (built-in):
 - 'Apple'
 - 'Google'
 - 'Microsoft'
 - 'tech giants'
2. Sử dụng hàm `extract_noun_chunks_custom`:
 - 'Apple , Google and Microsoft'
 - 'tech giants'
3. Sử dụng hàm `extract_noun_chunks_simple`:
 - 'Apple'
 - 'Google'
 - 'Microsoft'
 - 'tech'
 - 'tech giants'

4. Câu: 'The CEO of the company announced new products.'

1. Sử dụng `.noun_chunks` (built-in):
 - 'The CEO'
 - 'the company'
 - 'new products'
2. Sử dụng hàm `extract_noun_chunks_custom`:

- 'The CEO of the company'
- 'new products'

3. Sử dụng hàm `extract_noun_chunks_simple`:

- 'The CEO'
- 'the company'
- 'new products'

Bài 3: Tìm đường đi ngắn nhất trong cây

Mục tiêu Viết hàm `get_path_to_root(token)` để tìm đường đi từ một token bất kỳ lên đến gốc (ROOT) của cây, và hàm `get_path_between_tokens(token1, token2)` để tìm đường đi ngắn nhất giữa hai token.

Lý thuyết Cây phụ thuộc có cấu trúc phân cấp với ROOT ở đỉnh. Mỗi token có một head (từ điều khiển) và có thể có nhiều dependents (từ phụ thuộc). Đường đi giữa hai token trong cây có thể được tìm thấy thông qua:

Tìm đường từ mỗi token lên ROOT

Tìm điểm giao nhau gần nhất (lowest common ancestor)

Kết hợp hai đoạn đường đi

```
In [18]: def get_path_to_root(token):
    """
    Tìm đường đi từ một token lên đến gốc (ROOT) của cây

    Args:
        token: Token cần tìm đường đi lên root

    Returns:
        list: Danh sách các token trên đường đi từ token đến ROOT
    """
    path = [token]
    current_token = token

    # Đi lên dần qua các head cho đến khi gặp ROOT
    while current_token.dep_ != "ROOT" and current_token.head != current_token:
        current_token = current_token.head
        path.append(current_token)

    return path

def get_path_between_tokens(token1, token2):
    """
    Tìm đường đi ngắn nhất giữa hai token trong cây phụ thuộc

    Args:
        token1, token2: Hai token cần tìm đường đi
```

```

Returns:
    list: Danh sách các token trên đường đi ngắn nhất
    """
    path1_to_root = get_path_to_root(token1)

    path2_to_root = get_path_to_root(token2)

    # Tìm điểm giao nhau gần nhất (lowest common ancestor)
    common_ancestor = None
    for i, t1 in enumerate(path1_to_root):
        for j, t2 in enumerate(path2_to_root):
            if t1 == t2:
                common_ancestor = t1
                path1_part = path1_to_root[:i+1]
                path2_part = path2_to_root[:j]
                path2_part.reverse()

                # Kết hợp hai phần
                full_path = path1_part + path2_part
                return full_path

    return []

```

```

In [21]: def find_tokens_by_text(doc, word):
    """
    Tìm tất cả token có text trùng với word trong doc
    """
    return [token for token in doc if token.text.lower() == word.lower()]

def print_path_info(doc, start_word, end_word=None):
    """
    In thông tin đường đi
    """
    # Tìm token bắt đầu
    start_tokens = find_tokens_by_text(doc, start_word)

    if not start_tokens:
        print(f"Không tìm thấy từ '{start_word}' trong câu")
        return

    start_token = start_tokens[0]

    print(f"Duong di tu '{start_token.text}' len ROOT:")
    path_to_root = get_path_to_root(start_token)

    path_display = []
    for i, token in enumerate(path_to_root):
        if i == 0:
            path_display.append(f"BAT DAU: {token.text}")
        elif token.dep_ == "ROOT":
            path_display.append(f"ROOT: {token.text}")
        else:
            path_display.append(f"{token.text} ({token.dep_})")

```

```

print(" -> ".join(path_display))

if end_word:
    end_tokens = find_tokens_by_text(doc, end_word)

    if end_tokens:
        end_token = end_tokens[0]
        print(f"\nDuong di tu '{start_token.text}' den '{end_token.text}':")
        path_between = get_path_between_tokens(start_token, end_token)

        if path_between:
            path_between_display = []
            for i, token in enumerate(path_between):
                if i == 0:
                    path_between_display.append(f"BAT DAU: {token.text}")
                elif i == len(path_between) - 1:
                    path_between_display.append(f"KET THUC: {token.text}")
                elif token.dep_ == "ROOT":
                    path_between_display.append(f"ROOT: {token.text}")
                else:
                    path_between_display.append(f"{token.text} ({token.dep_}")

            print(" -> ".join(path_between_display))
        else:
            print("Khong tim thay duong di")
    else:
        print(f"Khong tim thay tu '{end_word}' trong cau")

```

```

In [ ]: sentence3 = "I want to buy a new computer for programming."
doc3 = nlp(sentence3)

print(f"Cau: '{sentence3}'")

words_to_check = ["I", "buy", "computer", "programming"]
for word in words_to_check:
    tokens = find_tokens_by_text(doc3, word)
    if tokens:
        token = tokens[0]
        path = get_path_to_root(token)
        path_text = [f"{t.text}({t.dep_})" for t in path]
        print(f"'{word}' -> ROOT: {' -> '.join(path_text)}")

```

```

Cau: 'I want to buy a new computer for programming.'
'I' -> ROOT: I(nsubj) -> want(ROOT)
'buy' -> ROOT: buy(xcomp) -> want(ROOT)
'computer' -> ROOT: computer(dobj) -> buy(xcomp) -> want(ROOT)
'programming' -> ROOT: programming(pobj) -> for(preposition) -> buy(xcomp) -> want(ROOT)

```

Kết luận

Ba bài tập này giúp:

Hiểu sâu về cấu trúc cây phụ thuộc và nắm vững cách truy cập và duyệt cây theo chương trình ứng dụng vào các bài toán thực tế trong xử lý ngôn ngữ tự nhiên có kỹ năng phân tích cú pháp phụ thuộc là nền tảng cho nhiều ứng dụng NLP nâng cao như trích xuất thông tin, phân tích tình cảm, dịch máy và hệ thống hỏi đáp.

Bài 1: Tìm động từ chính (find_main_verb)

- Hiểu được ROOT là trung tâm của câu
- Biết cách xác định động từ chính trong mọi loại câu

Bài 2: Trích xuất cụm danh từ (extract_noun_chunks)

- Hiểu cấu trúc của cụm danh từ
- Biết cách duyệt cây để thu thập modifiers
- So sánh với phương pháp built-in của spaCy

Bài 3: Tìm đường đi trong cây (get_path_to_root, get_path_between_tokens)

- Hiểu cấu trúc phân cấp của cây phụ thuộc
- Biết cách tìm common ancestor
- Ứng dụng trong tìm quan hệ giữa các từ

Ứng dụng thực tế:

1. Phân tích câu hỏi: "Ai làm gì?" (S-V-O)
2. Trích xuất thông tin từ văn bản
3. Phân tích quan hệ ngữ nghĩa
4. Hỗ trợ dịch máy và tóm tắt văn bản ("")